MLOps Practical 9: Open-Source Chatbot Development Using Rasa

We'll perform Practical 9 from the beginning in your Windows PowerShell, using your existing Rasa virtual environment. I'll give you one step at a time, with commands in the correct order.

Aim: To develop and test an open-source chatbot using Rasa that understands user messages and responds with suitable answers.

Important: Your earlier error occurred because `config.yml` and the `models` folder were missing. We'll create the project files before training or running the chatbot.

Step 1: Open your project folder

Open PowerShell and run these commands:

```
cd C:\Users\ACER\mlops_practical\practical2\RasaChatbot
.\venv\Scripts\Activate.ps1
```

Your terminal should start with `(venv)`, like this:

```
(venv) PS C:\Users\ACER\mlops_practical\practical2\RasaChatbot>
```

Check your Rasa installation:

```
rasa --version
```

If a version is displayed, continue.

Step 2: Create the Rasa chatbot project

First, check the files in your folder:

```
dir
```

If `config.yml`, `domain.yml`, `data`, and `models` are already present, don't initialize the project again; go to Step 4.

If the project files are missing, run:

```
rasa init --no-prompt
```

Wait for the command to finish. It should generate the default project files and train an initial model.

Step 3: Verify the project files

After initialization finishes, run:

```
dir
```

You should see files similar to these:

```
config.yml
credentials.yml
domain.yml
endpoints.yml
data
models
```

The important files are:

- `config.yml` — defines how the chatbot processes language.
- `domain.yml` — defines intents, responses, and conversation details.
- `data` — contains training examples and conversation rules.
- `models` — stores the trained chatbot model.

Step 4: Train the chatbot

Run:

```
rasa train
```

Wait until training completes. Rasa should save a model in the `models` folder.

You may see deprecation warnings about SQLAlchemy or `pkg_resources`. These warnings alone do not mean the practical failed. Look for an actual error or a successful training message.

Step 5: Start the chatbot

Run:

```
rasa shell
```

When the chatbot starts, enter the following messages one at a time:

```
hello
what is your name?
goodbye
```

The default Rasa project includes sample conversation data, so the exact response may vary depending on the trained model.

To exit the chatbot, press Ctrl + C.

Step 6: Verify the trained model

Exit the chatbot and run:

```
dir models
```

You should see a trained model file, usually ending in `.tar.gz`.

You can also validate the training data with:

```
rasa data validate
```

If validation completes without errors, your training data is valid.

Step 7: Record the result in your practical journal

Result: An open-source chatbot was created using Rasa. The project was initialized, the training data was processed, the model was trained, and the chatbot was tested using the command-line interface.